
● SAVOL 1:

PYQT5 KUTUBXONASINING ASOSIY IMKONIYATLARI VA ARXITEKTURASI. SIGNAL-SLOT MEXANIZMI QANDAY ISHLAYDI VA U AN'ANAVIY CALLBACK USULIDAN NIMASI BILAN FARQ QILADI? REAL DASTURLASHDA QANDAY FOYDA BERADI?

1. PyQt5 nima?

PyQt5 — bu Qt5 (C++ da yozilgan kuchli GUI framework) ning Python uchun binding (ko'prigi). Ya'ni C++ da yozilgan Qt kutubxonasini Python orqali ishlatish imkonini beradi.

Qt5 (C++) ← PyQt5 (Python binding) ← Sizning Python kodingiz

2. PyQt5 ning asosiy imkoniyatlari

GUI Komponentlar (Widgets)

```
from PyQt5.QtWidgets import *
```

Asosiy widgetlar

QPushButton	<i># Tugma</i>
QLabel	<i># Matn/rasm ko'rsatish</i>
QLineEdit	<i># Bir qatorli input</i>
QTextEdit	<i># Ko'p qatorli input</i>
QComboBox	<i># Dropdown ro'yxat</i>
QCheckBox	<i># Belgilash qutisi</i>
QRadioButton	<i># Radio tugma</i>
QSlider	<i># Siljitgich</i>
QProgressBar	<i># Progress bar</i>
QListWidget	<i># Ro'yxat</i>
QTableWidget	<i># Jadval</i>
QTreeWidget	<i># Daraxt ko'rinish</i>

Layout Menejerlari

QHBoxLayout	<i># Gorizontal joylashtirish</i>
QVBoxLayout	<i># Vertikal joylashtirish</i>
QGridLayout	<i># Setka joylashtirish</i>
QFormLayout	<i># Form ko'rinishi</i>
QStackedLayout	<i># Qatlamli joylashtirish</i>

PyQt'da **Layout (joylashtirish) menejerlari** oynadagi vidjetlar (tugmalar, matn maydonlari, rasmlar va h.k.) qanday joylashishini va oyna o'lchami o'zgarganda ularning o'lchamlari qanday moslashishini boshqaradi.

Siz sanab o'tgan 5 ta asosiy layout'ning har biri o'ziga xos vazifani bajaradi. Keling, ularning farqlarini aniq va hayotiy misollar yordamida ko'rib chiqamiz.

1. QHBoxLayout (Gorizontal joylashtirish)

Bu layout vidjetlarni chapdan o'ngga qarab, **yonma-yon (gorizontal)** qatorda joylashtiradi. Agar oyna kengaytirilsa, vidjetlar orasidagi bo'shliq ham shunga mos ravishda kengayadi.

- **Konkret misol:** Dastur oynasining eng pastki qismida joylashgan tugmalar paneli. Tasavvur qiling, sizda **"Saqlash" (Save)** va **"Bekor qilish" (Cancel)** tugmalari bor. Ular yonma-yon turishi kerak. Shunda siz `HBoxLayout` dan foydalanasiz.
- **Ko'rinishi:** [Tugma 1] [Tugma 2] [Tugma 3]

2. QVBoxLayout (Vertikal joylashtirish)

Bu layout vidjetlarni yuqoridan pastga qarab, **ustma-ust (vertikal)** ustun shaklida joylashtiradi.

- **Konkret misol:** Oddiy ro'yxat yoki menyu. Masalan, Telegram'ning sozlamalar bo'limini tasavvur qiling: "Profil", "Xabarnomalar", "Xavfsizlik", "Til" kabi tugmalar ketma-ket, bir-birining tagida joylashgan. Buni `VBoxLayout` yordamida qilish juda oson. Yoki oddiy MP3 pleyer: yuqorida rasm, uning tagida qo'shiq nomi, eng pastda esa boshqaruv tugmalari (Play/Pause).
- **Ko'rinishi:**

[Vidjet 1]

[Vidjet 2]

[Vidjet 3]

3. QGridLayout (Setka / To'rsimon joylashtirish)

Bu layout vidjetlarni **qatorlar va ustunlar** ga (jadvalga o'xshab) bo'lib joylashtiradi. Siz vidjetni aynan qaysi qator va qaysi ustunda joylashishini (koordinatalarini) aniq ko'rsata olasiz. Shuningdek, bitta vidjet bir nechta katakni egallashi ham mumkin (masalan, ikkita ustunni birlashtirish).

- **Konkret misol: Kalkulyator interfeysi.** Kalkulyatorda raqamlar va amallar (0-9, +, -, *, /) 4x4 yoki 5x4 o'lchamdagi aniq setka shaklida joylashgan bo'ladi. Yoki "Shaxmat" doskasi ko'rinishidagi o'yin maydoni.
- **Ko'rinishi:**

[7] [8] [9] [/]

[4] [5] [6] [*]

[1] [2] [3] [-]

4. QFormLayout (Forma ko'rinishida joylashtirish)

Bu layout maxsus ikki ustunli tizim bo'lib, odatda **ma'lumot kiritish formalarini** yaratish uchun ishlatiladi. Chap ustunda vidjetning nomi (Label), o'ng ustunda esa ma'lumot kiritish maydoni (LineEdit, ComboBox va h.k.) joylashadi.

- **Konkret misol: Ro'yxatdan o'tish (Registration) yoki Login formasi.** Dastur foydalanuvchidan o'z ma'lumotlarini kiritishni so'raganda ishlatiladi. Garchi buni `QGridLayout` yordamida ham qilsa bo'lsa-da, `QFormLayout` bu jarayonni avtomatlashtiradi va kodni qisqartiradi.
- **Ko'rinishi:**

Ism: [_____]

Familiya: [_____]

Parol: [_____]

5. `QStackedLayout` (Qatlamli / Ustma-ust joylashtirish)

Bu layout vidjetlarni huddi kartalar kabi **ustma-ust qatlamlar** shaklida joylashtiradi. Eng muhim farqi: bir vaqtning o'zida ekranda **faqat bitta qatlam (bitta vidjet/oyna) ko'rinib turadi**. Boshqa qatlamlarga o'tish uchun maxsus tugmalar (yoki ComboBox) kerak bo'ladi.

- **Konkret misol: Dastur o'rnatish ustasi (Installation Wizard).** Odatda dastur o'rnatayotganda "Next" (Keyingisi) tugmasini bosasiz, shunda oyna ichidagi ma'lumotlar o'zgaradi, lekin asosiy oyna ramkasi joyida qoladi. Yana bir misol: brauzerlardagi "Tab"lar (garchi tablar uchun tayyor `QTabWidget` bo'lsa ham, u ichkarida aynan mana shu Stacked tizimidan foydalanadi).

Xulosa qilib aytganda:

Agar sizga vidjetlarni:

- Yonga terish kerak bo'lsa → **`QHBoxLayout`**
- Pastga terish kerak bo'lsa → **`QVBoxLayout`**
- Katakchalarga aniq joylash kerak bo'lsa → **`QGridLayout`**
- So'rovnomani/anketa yasash kerak bo'lsa → **`QFormLayout`**
- Bir xil joyda turli xil sahifalarni almashtirib ko'rsatish kerak bo'lsa → **`QStackedLayout`** dan foydalanasiz.

Katta va murakkab dasturlarda bu layoutlar **bir-birining ichiga joylashtirilib (nested)** ishlatiladi.

Masalan, butun oyna `QVBoxLayout` da bo'lib, uning eng pastki qismidagi tugmalar `QHBoxLayout` ga solinishi mumkin.



Oyna Turlari

```
QMainWindow    # Asosiy oyna (menu, toolbar bilan)
QDialog        # Dialog oyna
QWidget        # Oddiy widget/oyna
QMessageBox    # Xabar oynasi
QFileDialog    # Fayl tanlash oynasi
QInputDialog   # Input dialog
```



1-qism: Qt Designer'dagi 5 xil boshlang'ich oyna (Shablonlar)

Qt Designer sizga dastur yaratishni osonlashtirish uchun quyidagi tayyor qoliplarni beradi:

1. Main Window (Asosiy oyna):

- **Nima u?** Eng to'liq va yirik oyna qolipi. U o'zida avtomatik ravishda Menyu qatori (Menu Bar), Uskunalar paneli (Tool Bar) va Holat qatori (Status Bar) ni saqlaydi.
- **Misol:** Microsoft Word yoki Google Chrome kabi to'laqonli dasturlarning asosiy oynasi. Yuqorida "File", "Edit" menyulari bo'ladi.

2. Widget (Oddiy vidjet):

- **Nima u?** Hech qanday menyu yoki maxsus qatorlarsiz, mutlaqo bo'm-bo'sh oyna. U oq qog'ozga o'xshaydi, ichiga xohlagan narsangizni tashlaysiz.
- **Misol:** Kichikroq, oddiy dasturlar (masalan, kalkulyator) yoki kattaroq oynaning ichiga o'rnatiladigan alohida qism (masalan, Telegram'dagi chat qismi alohida bir Widget hisoblanadi).

3. Dialog with Buttons Bottom (Pastida tugmalari bor dialog):

- **Nima u?** Odatda foydalanuvchidan qandaydir tasdiq so'raladigan oyna. Uning pastki qismida avtomatik ravishda `OK` va `Cancel` tugmalari joylashgan bo'ladi.
- **Misol:** Dastur sozlamalarini (Settings) o'zgartirib bo'lgach, saqlashni so'raydigan oyna.

4. Dialog with Buttons Right (O'ngida tugmalari bor dialog):

- **Nima u?** Yuqoridagining aynan o'zi, faqat `OK` va `Cancel` tugmalari oynaning o'ng tomonida vertikal joylashgan bo'ladi (bu ko'proq macOS yoki ba'zi Linux tizimlari uslubiga xos).

5. Dialog without Buttons (Tugmalarsiz dialog):

- **Nima u?** Bu ham qo'shimcha muloqot oynasi (Dialog), lekin unga hech qanday tayyor tugma qo'shilmagan. Tugmalarni o'zingiz xohlagan joyga joylashtirishingiz mumkin.



2-qism: PyQt5 dagi Asosiy Oyna Turlari

Endi kod darajasidagi sinflarga (class) o'tamiz. Bu sinflar oyna qanday ishlashini belgilaydi.

1. QMainWindow (Asosiy oyna)

Bu dasturning "yuragi". U faqat bitta bo'ladi (keng qamrovli dasturlarda). Uning o'ziga xos arxitekturasida bor: markazda asosiy ish maydoni (`Central Widget`) bo'ladi, atrofida esa menyular.

- **Konkret misol:** Fotoshop dasturi. Tepada menyular (`File` , `Filter`), chapda asboblar paneli (`Toolbar`), pastda holat paneli (masalan, rasm o'lchami ko'rinib turadi) va o'rtada siz chizayotgan rasm (`Central Widget`).

2. QDialog (Dialog oyna)

Foydalanuvchi bilan qisqa muddatli muloqot qilish uchun asosiy oynaning ustidan qalqib chiqadigan (pop-up) oyna. U ko'pincha *Modal* bo'ladi (ya'ni, dialog oynasini yopmaguningizcha, orqadagi asosiy oynani bosolmaysiz).

- **Konkret misol:** Telegram'da "Add Contact" (Kontakt qo'shish) tugmasini bossangiz, ism va raqam kiritish uchun kichik oyna chiqadi. Shu oyna – `QDialog`.

3. `QWidget` (Oddiy widget)

Bu PyQt'dagi barcha vizual elementlarning "otasi". Tugma (`QPushButton`) ham, matn maydoni (`QLineEdit`) ham aslida `QWidget` dan tarqalgan. Agar `QWidget` ni boshqa oynaning ichiga solmasangiz, u mustaqil oyna sifatida ochiladi. U juda universal.

- **Konkret misol:** Dastur ichida bir nechta tablar (varaqlar) bo'lsa, har bir tabning ichidagi kontent alohida `QWidget` hisoblanadi.

4. `QMessageBox` (Xabar oynasi)

Bu `QDialog` ning tayyor va maxsus turi. Uni noldan yasash shart emas, chunki tizimda tayyor shabloni bor. U faqat ma'lumot berish, ogohlantirish yoki xato haqida xabar berish uchun ishlatiladi.

- **Konkret misol:** Hujjatni saqlamasdan yopmoqchi bo'lsangiz chiqadigan: "*Hujjat saqlanmadi. Saqlashni xohlaysizmi? [Yes] [No]*" oynasi. Yoki "*Parol noto'g'ri kiritildi*" degan xato (Error) oynasi.

5. `QFileDialog` (Fayl tanlash oynasi)

Bu ham tayyor oyna. Operatsion tizimning o'zida bor bo'lgan kompyuter ichidan fayl qidirish oynasini chaqirib beradi.

- **Konkret misol:** Dasturda "Open" (Ochish) tugmasini bosganingizda kompyuteringizdagi papkalar ro'yxati chiqib, kerakli rasm yoki faylni tanlashingizni so'raydigan oyna.

6. `QInputDialog` (Input dialog)

Foydalanuvchidan bittagina ma'lumot (matn, raqam yoki ro'yxatdan birini tanlash) so'rash uchun mo'ljallangan juda sodda tayyor oyna.

- **Konkret misol:** Wi-Fi tarmog'iga ulanmoqchi bo'lganingizda, faqatgina "*Parolni kiriting: [_____]*" deb chiqadigan kichkina oyna.

3-qism: Bu ikki ro'yxatning o'zaro bog'liqligi

Qt Designer'dagi 5 ta shablon – bu shunchaki vizual qulaylik. Ular aslida PyQt5'dagi kod sinflarining ko'rinishidir. Ularning bog'liqligi quyidagicha:

Qt Designer'dagi Shablon nomi	PyQt5 dagi Class (Sinf) nomi	Izoh
Main Window	<code>QMainWindow</code>	To'liq menyuli asosiy oyna kodi hosil bo'ladi.
Widget	<code>QWidget</code>	Bo'sh, oddiy oyna kodi hosil bo'ladi.
Dialog (Buttons Bottom/Right)	<code>QDialog</code>	<code>QDialog</code> yaratiladi va ichiga <code>QDialogButtonBox</code> (OK/Cancel tugmalari) o'rnatiladi.
Dialog without Buttons	<code>QDialog</code>	Bo'm-bo'sh <code>QDialog</code> yaratiladi.

Diqqat qiling: E'tibor bersangiz, Qt Designer'da **QMessageBox**, **QFileDialog** yoki **QInputDialog** uchun tayyor shablon yo'q! **Nega?** Chunki bu uchalasi allaqachon tayyor va mukammal yozilgan (built-in) oynalardir. Ularni Qt Designer'da chizib, dizayn qilib o'tirishning keragi yo'q. Ular shunchaki Python kodining ichida bir qator kod bilan (masalan, `QFileDialog.getOpenFileName()`) chaqirib ishlatiladi. Designer esa faqat siz o'zingiz noldan chizishingiz (dizayn qilishingiz) kerak bo'lgan oynalar (`QMainWindow`, `QWidget`, `QDialog`) uchun ishlatiladi.

Grafika va Multimedia

```
from PyQt5.QtGui import *
QPainter          # 2D chizish
QPixmap           # Rasm
QFont              # Shrift
QColor             # Rang
QIcon              # Ikonka

from PyQt5.QtMultimedia import *
QMediaPlayer       # Video/audio ijro
```

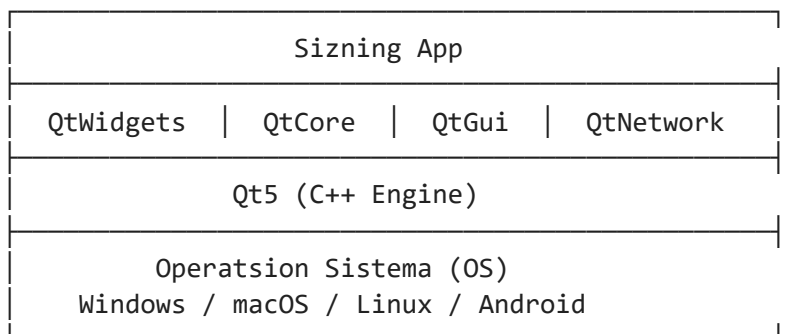
Tarmoq

```
from PyQt5.QtNetwork import *
QTcpSocket         # TCP ulanish
QUdpSocket         # UDP ulanish
QTcpServer         # TCP server
QNetworkAccessManager # HTTP so'rovlar
```

Boshqa Imkoniyatlar

```
from PyQt5.QtCore import *
QTimer             # Taymer
QThread            # Ko'p oqimlilik
QFile              # Fayl operatsiyalari
QSettings          # Sozlamalarni saqlash
QTranslator        # Tarjima (i18n)
```

3. PyQt5 Arxitekturası



Asosiy Modullar:

Modul	Vazifasi
QtWidgets	Barcha UI elementlar
QtCore	Signal/slot, timer, thread, fayl
QtGui	Shrift, rang, rasm, chizish
QtNetwork	Tarmoq operatsiyalari
QtSql	Ma'lumotlar bazasi
QtWebEngineWidgets	Web brauzer
QtMultimedia	Audio/video

4. Event Loop — Dasturning Yuragi

```
from PyQt5.QtWidgets import QApplication, QMainWindow
import sys
```

```
app = QApplication(sys.argv) # 1. App yaratiladi
window = QMainWindow()      # 2. Oyna yaratiladi
window.show()                # 3. Ko'rsatiladi
sys.exit(app.exec_())        # 4. Event loop boshlanadi ← muhim!
```

Event Loop qanday ishlaydi:

```

Foydalanuvchi harakat qiladi (klik, klaviatura...)
↓
OS event yaratadi
↓
Qt event queue ga tushadi
↓
app.exec_() ushlab oladi
↓
Tegishli widget ga yuboradi
↓
Signal emit bo'ladi → Slot chaqiriladi

```

5. Signal-Slot Mexanizmi — Eng Muhim Qism!

Tushuncha:

```

SIGNAL = "Biror narsa sodir bo'ldi!" degan xabar
SLOT   = O'sha xabarga javob beradigan funksiya
CONNECT = Ikkalasini bog'laydigan ko'pri

```

Sodda misol:

```
from PyQt5.QtWidgets import *
import sys

class MisalDarsa(QWidget):
```

```

def __init__(self):
    super().__init__()

    # Tugma yaratish
    self.tugma = QPushButton("Bosing!", self)
    self.tugma.setGeometry(50, 50, 200, 50)

    # SIGNAL → SLOT bog'lash
    self.tugma.clicked.connect(self.tugma_bosildi)
    #           ↑ signal      ↑ connect  ↑ slot (funksiya)

def tugma_bosildi(self): # ← Bu SLOT
    print("Tugma bosildi!")

app = QApplication(sys.argv)
darsa = MisalDarsa()
darsa.show()
sys.exit(app.exec_())

```

Turli Signal turlari:

```

# Parametrsiz signal
self.tugma.clicked.connect(self.funksiya)

# Parametrli signal (bool)
self.checkbox.stateChanged.connect(self.holat_ozgardi)
def holat_ozgardi(self, holat: bool):
    print(f"Holat: {holat}")

# Parametrli signal (str)
self.lineEdit.textChanged.connect(self.matn_ozgardi)
def matn_ozgardi(self, matn: str):
    print(f"Matn: {matn}")

# Parametrli signal (int)
self.slider.valueChanged.connect(self.qiymat_ozgardi)
def qiymat_ozgardi(self, qiymat: int):
    print(f"Qiymat: {qiymat}")

```

PyQt dagi slider (kodda `QSlider` deb ataladi) — bu foydalanuvchiga ma'lum bir raqamlar oraliqdan (masalan, 0 dan 100 gacha) kerakli qiymatni sichqoncha bilan surish orqali tanlash imkonini beruvchi grafik interfeys (GUI) elementidir.

Buni kundalik hayotda kompyuter yoki telefonda **ovoz balandligini sozlash** yoki videoni **oldinga/orqaga o'tkazish** uchun ishlatiladigan surma tugmalariga o'xshatishingiz mumkin.

`QSlider` ning Asosiy Xususiyatlari

Slider bilan ishlashda eng ko'p ishlatiladigan xususiyatlar va funksiyalar:

- **Yo'nalish (Orientation):** Slider yotiq (`Qt.Horizontal`) yoki tikka (`Qt.Vertical`) bo'lishi mumkin.
- **Minimum va Maksimum qiymatlar:** Slider'ning eng kichik va eng katta qiymatlarini belgilash (masalan, `setMinimum(0)` va `setMaximum(100)`).

- **Boshlang'ich qiymat:** Dastur ishga tushganda slider qaysi joyda turishini belgilash (`setValue(50)`).
 - **Signal (Hodisa):** Foydalanuvchi sliderni surganda, u `valueChanged` degan signalni yuboradi. Buni biror funksiyaga ulab (`connect`), qiymat o'zgarganda ekrandagi matnni yoki o'zgaruvchini yangilash mumkin.
 - **Chiziqchalar (Ticks):** Slider ostida yoki yonida yo'naltiruvchi chiziqchalarni ko'rsatish (`setTickPosition()`).
-

PyQt'da QSlider uchun Sodda Misol

Quyida PyQt5 yordamida yozilgan sodda dastur kodi keltirilgan. Unda slider va bitta matn (Label) bor. Sliderni sursangiz, matndagi raqam o'zgaradi.

```
import sys
from PyQt5.QtWidgets import QApplication, QWidget, QSlider, QVBoxLayout, QLabel
from PyQt5.QtCore import Qt

class SliderIlovasi(QWidget):
    def __init__(self):
        super().__init__()

        # Oyna sozlamalari
        self.setWindowTitle("QSlider Misoli")
        self.setGeometry(200, 200, 350, 150)

        # Layout (Elementlarni joylashtirish tartibi)
        layout = QVBoxLayout()

        # Matn (Label) yaratish
        self.label = QLabel("Joriy qiymat: 50", self)
        self.label.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.label)

        # Yotiq (Horizontal) Slider yaratish
        self.slider = QSlider(Qt.Horizontal, self)
        self.slider.setMinimum(0)          # Eng kam qiymat
        self.slider.setMaximum(100)        # Eng ko'p qiymat
        self.slider.setValue(50)           # O'rtada turishi uchun 50 berildi

        # Slider ostida chiziqchalar ko'rsatish
        self.slider.setTickPosition(QSlider.TicksBelow)
        self.slider.setTickInterval(10)    # Har 10 qadamda bitta chiziq

        # Slider qiymati o'zgarganda update_label funksiyasini ishga tushirish
        self.slider.valueChanged.connect(self.update_label)

        layout.addWidget(self.slider)
        self.setLayout(layout)

        # Slider qiymati o'zgarganda ishlaydigan funksiya
        def update_label(self, value):
            self.label.setText(f"Joriy qiymat: {value}")

if __name__ == '__main__':
```

```
app = QApplication(sys.argv)
oyna = SliderIlovasi()
oyna.show()
sys.exit(app.exec_())
```

Kod qanday ishlaydi?

1. `QSlider(Qt.Horizontal)` orqali biz o'ngga va chapga suriladigan yotiq slider yaratdik.
2. `setMinimum(0)` va `setMaximum(100)` orqali foydalanuvchi faqat 0 dan 100 gacha bo'lgan raqamlarni tanlay olishini belgiladik.
3. Asosiy qism — `self.slider.valueChanged.connect(self.update_label)`. Bu qator orqali biz dasturga: "Agar slider o'rnidan jilsa, darhol o'sha yangi raqamni olib `update_Label` funksiyasiga ber", deb buyruq beryapmiz. U funksiya esa yozuvni yangilaydi.

PyQt'da eng ko'p ishlatiladigan vizual elementlar (vidjetlar) va ularning asosiy signallari:

- **QPushButton (Tugma)**
 - `clicked` — Tugma bosilganda jo'natiladi.
- **QLineEdit (Matn kiritish maydoni)**
 - `textChanged` — Maydondagi matn o'zgarganda (har bir harf yozilganda yoki o'chirilganda).
 - `returnPressed` — Kiritish maydonida "Enter" tugmasi bosilganda.
- **QComboBox (Ochiluvchi ro'yxat)**
 - `currentIndexChanged` — Ro'yxatdan boshqa variant (element) tanlanganda.
- **QCheckBox (Belgilash katakchasi / Ptichka)**
 - `stateChanged` — Katakchaga belgi qo'yilganda yoki olib tashlanganda.
- **QSpinBox (Raqam kiritish qutisi)**
 - `valueChanged` — Raqam qiymati o'zgarganda (xuddi slider kabi ishlaydi).
- **QRadioButton (Yakka tanlov tugmasi)**
 - `toggled` — Tugma holati yoqilganda yoki o'chirilganda.

Bu signallarning barchasi kodda `element.signal_nomi.connect(funksiya)` shaklida kerakli jarayonga ulanadi.

6. O'zingizning Signal yaratish

```
from PyQt5.QtCore import QObject, pyqtSignal

class Hisoblagich(QObject):

    # O'z signalimizni e'lon qilamiz
```

```

    qiymat_ozgardi = pyqtSignal(int)           # int yuboradi
    matn_yuborildi = pyqtSignal(str)          # str yuboradi
    togri_javob    = pyqtSignal(int, str)      # int va str yuboradi
    oddiy_signal    = pyqtSignal()             # hech narsa yubormaydi

    def __init__(self):
        super().__init__()
        self.qiymat = 0

    def oshir(self):
        self.qiymat += 1
        self.qiymat_ozgardi.emit(self.qiymat) # ← signalni "otish"
        self.matn_yuborildi.emit(f"Qiymat: {self.qiymat}")

# Ishlatish:
hisoblagich = Hisoblagich()
hisoblagich.qiymat_ozgardi.connect(lambda x: print(f"Yangi qiymat: {x}"))
hisoblagich.matn_yuborildi.connect(print)

hisoblagich.oshir() # "Yangi qiymat: 1" va "Qiymat: 1"
hisoblagich.oshir() # "Yangi qiymat: 2" va "Qiymat: 2"

```

7. Signal-Slot va Callback — Farqi nima?

Callback usuli (an'anaviy):

❌ *Callback – eski, chalkash usul*

```

class Tugma:
    def __init__(self):
        self.callback = None # faqat 1 ta funksiya!

    def bosildi(self):
        if self.callback:
            self.callback() # faqat bittasini chaqiradi

class Dastur:
    def __init__(self):
        self.tugma = Tugma()
        self.tugma.callback = self.birinchi_ish # faqat 1 ta bog'lash!

    def birinchi_ish(self):
        print("Birinchi ish")

# ❌ Muammo: ikkinchi ish ham kerak bo'lsa?
# self.tugma.callback = self.ikkinchi_ish ← birinchisi o'chadi!

```

Signal-Slot usuli (PyQt5):

✅ *Signal-Slot – zamonaviy, kuchli usul*

```

class Dastur(QWidget):
    def __init__(self):
        super().__init__()

```

```

self.tugma = QPushButton("Bosing!")

# ✅ Ko'plab slotlar bog'lash mumkin!
self.tugma.clicked.connect(self.birinchi_ish)
self.tugma.clicked.connect(self.ikkinchi_ish)
self.tugma.clicked.connect(self.uchinchi_ish)
# Hammasi ishlaydi!

def birinchi_ish(self):
    print("Birinchi!")

def ikkinchi_ish(self):
    print("Ikkinchi!")

def uchinchi_ish(self):
    print("Uchinchi!")

```

8. Callback vs Signal-Slot — To'liq Taqqoslash

Xususiyat	Callback	Signal-Slot
Bir signalga ko'p funksiya	❌ Yo'q	✅ Bor
Ob'ektlar orasida bog'lash	❌ Qiyin	✅ Oson
Thread xavfsizligi	❌ Yo'q	✅ Bor
Disconnect qilish	❌ Qiyin	✅ Oson
Kodni tartibli saqlash	❌ Chalkash	✅ Tartibli
Parametr yuborish	⚠️ Cheklangan	✅ Kuchli
Lambda ishlatish	⚠️ Qiyin	✅ Oson

9. Signal-Slot ning Real Afzalliklari

✅ 1. Disconnect — bog'liqlikni uzish

```

# Bog'lash
self.tugma.clicked.connect(self.funksiya)

# Uzish (kerak bo'lganda)
self.tugma.clicked.connect(self.funksiya)

# Barcha slotlarni uzish
self.tugma.clicked.disconnect()

```

✅ 2. Lambda bilan qisqa yozish

```

self.tugma.clicked.connect(lambda: print("Bosildi!"))
self.slider.valueChanged.connect(lambda v: self.label.setText(str(v)))

```

✅ 3. Thread-lararo xavfsiz aloqa

```

from PyQt5.QtCore import QThread, pyqtSignal

class UzunIsh(QThread):
    natija = pyqtSignal(str) # Thread dan UI ga xabar

    def run(self):
        # Og'ir hisoblash...
        import time
        time.sleep(3)
        self.natija.emit("Ish tugadi!") # ✅ xavfsiz!

class Oyna(QWidget):
    def __init__(self):
        super().__init__()
        self.thread = UzunIsh()

        # ✅ Thread signali → UI slot
        # Qt avtomatik thread-safe qiladi!
        self.thread.natija.connect(self.natijani_korsot)

    def natijani_korsot(self, xabar):
        print(xabar) # UI thread da xavfsiz ishlaydi

```

QThread nima?

QThread — bu PyQt'da **parallel (bir vaqtda) ishlaydigan alohida "ip"** (thread) yaratish uchun ishlatiladigan sinf.

Oddiy misol bilan tushuntirish

Tasavvur qiling: siz restoranda ovqat buyurtma qildingiz.

- **Asosiy thread** = ofitsiant (foydalanuvchi bilan gaplashadi, UI ni boshqaradi)
- **QThread** = oshpaz (orqa tomonda ovqat tayyorlaydi)

Agar oshpaz yo'q bo'lsa, ofitsiant ovqat tayyorlanguncha **hamma narsani to'xtatib** kutishi kerak edi. Bu esa dasturda **UI ning "muzlab" qolishiga** olib keladi.

Qachon kerak bo'ladi?

Holat	Muammo	Yechim
Katta fayl yuklash	UI muzlaydi	QThread
Internet so'rovi	Dastur javob bermaydi	QThread
Uzoq hisob-kitob	Tugmalar ishlamaydi	QThread

Sodda kod misoli

```
In [1]: from PyQt5.QtCore import QThread, pyqtSignal
from PyQt5.QtWidgets import QApplication, QMainWindow, QPushButton, QLabel
import time

# 1. QThread dan meros olamiz
class IshchiThread(QThread):
    natija = pyqtSignal(str) # Signal – tugaganda xabar yuboradi

    def run(self):
        # Bu qism orqa fonda ishlaydi
        time.sleep(3) # Og'ir ish simulyatsiyasi
        self.natija.emit("Ish tugadi! ✅")

# 2. Asosiy oyna
class Oyna(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("QThread misoli")
        self.resize(300, 150)

        self.label = QLabel("Tayyor", self)
        self.label.move(100, 40)

        self.tugma = QPushButton("Ishni boshlash", self)
        self.tugma.move(80, 80)
        self.tugma.clicked.connect(self.boshlash)

    def boshlash(self):
        self.label.setText("Ishlayapti... ⏳")
        self.thread = IshchiThread()
        self.thread.natija.connect(self.tugallandi) # Signal ulash
        self.thread.start() # Threadni ishga tushurish

    def tugallandi(self, xabar):
        self.label.setText(xabar)

app = QApplication([])
oina = Oyna()
oina.show()
app.exec_()
```

Out[1]: 0

Asosiy tushunchalar

QThread	
├─ run()	→ orqa fonda bajariladigan kod shu yerda
├─ start()	→ threadni ishga tushiradi
├─ quit()	→ threadni to'xtatadi
└─ pyqtSignal	→ thread → UI ga ma'lumot yuborish yo'li

Muhim qoida: Thread ichidan to'g'ridan-to'g'ri UI ni o'zgartirish **mumkin emas** — buning uchun **Signal/Slot** mexanizmidan foydalaniladi.

✅ 4. Bir slot — ko'p signal

```

# Har xil signallar bitta slotga bog'lanishi mumkin
self.tugma1.clicked.connect(self.umumiy_slot)
self.tugma2.clicked.connect(self.umumiy_slot)
self.timer.timeout.connect(self.umumiy_slot)

def umumiy_slot(self):
    print("Qaysidir hodisa sodir bo'ldi!")

```

10. To'liq Amaliy Misol

```

from PyQt5.QtWidgets import *
from PyQt5.QtCore import QTimer, pyqtSignal, QObject
import sys

class Hisoblagich(QObject):
    """Model – faqat mantiq"""
    qiymat_ozgardi = pyqtSignal(int)
    chegara_yetdi = pyqtSignal()

    def __init__(self, chegara=10):
        super().__init__()
        self.qiymat = 0
        self.chegara = chegara

    def oshir(self):
        self.qiymat += 1
        self.qiymat_ozgardi.emit(self.qiymat)
        if self.qiymat >= self.chegara:
            self.chegara_yetdi.emit()

    def nolga_tushir(self):
        self.qiymat = 0
        self.qiymat_ozgardi.emit(self.qiymat)

class AsosiyOyna(QMainWindow):
    """View – faqat ko'rsatish"""

    def __init__(self):
        super().__init__()
        self.setWindowTitle("Signal-Slot Namuna")
        self.setFixedSize(300, 200)

        # Model yaratish
        self.hisoblagich = Hisoblagich(chegara=5)

        # UI yaratish
        markaziy = QWidget()
        self.setCentralWidget(markaziy)
        tartib = QVBoxLayout(markaziy)

        self.label = QLabel("Qiymat: 0", self)
        self.label.setStyleSheet("font-size: 24px; text-align: center;")

        oshir_tugma = QPushButton("➕ Oshir")

```

```

reset_tugma = QPushButton("🔄 Reset")

tartib.addWidget(self.label)
tartib.addWidget(oshir_tugma)
tartib.addWidget(reset_tugma)

# Signal → Slot bog'lash
oshir_tugma.clicked.connect(self.hisoblagich.oshir)
reset_tugma.clicked.connect(self.hisoblagich.nolga_tushir)

self.hisoblagich.qiyamat_ozgardi.connect(self.ekranni_yangilar)
self.hisoblagich.chegara_yetdi.connect(self.ogohlantir)

def ekranni_yangilar(self, yangi_qiyamat):
    self.label.setText(f"Qiyamat: {yangi_qiyamat}")

def ogohlantir(self):
    QMessageBox.warning(self, "Ogohlantirish", "Chegara yetdi!")
    self.hisoblagich.nolga_tushir()

app = QApplication(sys.argv)
oina = AsosiyOyna()
oina.show()
sys.exit(app.exec_())

```

11. Xulosa — Asosiy Fikrlar

PyQt5 → Python + Qt5 = kuchli GUI dasturlash

Signal = "Biror voqea sodir bo'ldi" xabari
 Slot = O'sha xabarga javob funksiyasi
 Connect = Ikkalasini bog'lash

Callback dan farqi:

- ✓ Ko'p slotga bog'lash mumkin
- ✓ Thread-safe
- ✓ Oson disconnect
- ✓ Kodni tartibli saqlaydi
- ✓ Ob'ektlar orasida yaxshi aloqa

💡 **Eng muhim qoida:** Signal-Slot — bu PyQt5 ning **qon tomiri**. Uni yaxshi tushungan odam PyQt5 ni to'liq tushungan hisoblanadi!

● SAVOL 2:

QLABEL VA QLINEEDIT VIDJETLARI O'RTASIDAGI FARQLAR. ULARNING XUSUSIYATLARI (SHRIFT, O'LCHAM, SETSTYLESHEET) VA FOYDALANISH HOLATLARI HAQIDA AMALIY MISOL BILAN TUSHUNTIRING.

1. Asosiy Farq — Bir Jumlada

QLabel = Faqat KO'RSATADI (foydalanuvchi o'zgartira olmaydi)
QLineEdit = Foydalanuvchi YOZADI va O'ZGARTIRADI

2. QLabel — To'liq Tavsif

QLabel — matn yoki rasm **ko'rsatish** uchun ishlatiladi. Foydalanuvchi u bilan faqat **o'qiy** oladi, yoza olmaydi.

Asosiy metodlar:

```
from PyQt5.QtWidgets import QLabel

label = QLabel("Salom Dunyo!")

# Matn
label.setText("Yangi matn")      # matn o'rnatish
label.text()                    # matnni olish

# Tekislash
from PyQt5.QtCore import Qt
label.setAlignment(Qt.AlignCenter)  # markazga
label.setAlignment(Qt.AlignRight)  # o'ngga
label.setAlignment(Qt.AlignLeft)   # chapga

# Rasm ko'rsatish
from PyQt5.QtGui import QPixmap
label.setPixmap(QPixmap("rasm.png"))  # rasm o'rnatish
label.setScaledContents(True)          # rasmni o'lchamga moslashtirish

# Wordwrap (ko'p qatorli matn)
label.setWordWrap(True)

# O'lcham
label.setFixedSize(200, 50)
label.setMinimumWidth(100)
label.setMaximumWidth(300)
```

QLabel signallari:

```
label.linkActivated.connect(...)  # HTML link bosilganda
label.linkHovered.connect(...)    # HTML link ustiga kelganda
```

⚠️ QLabel ning **o'z signali deyarli yo'q** — chunki u faqat ko'rsatadi!

3. QLineEdit — To'liq Tavsif

QLineEdit — foydalanuvchi **bir qator matn kiritishi** uchun ishlatiladi.

Asosiy metodlar:

```
from PyQt5.QtWidgets import QLineEdit

lineedit = QLineEdit()

# Matn
lineedit.setText("Boshlang'ich matn") # matn o'rnatish
lineedit.text()                       # kiritilgan matnni olish
lineedit.clear()                      # tozalash
lineedit.setPlaceholderText("Ismingiz") # bo'sh paytdagi ko'rsatma

# Parol rejimi
from PyQt5.QtWidgets import QLineEdit
lineedit.setEchoMode(QLineEdit.Password) # *** ko'rsatadi
lineedit.setEchoMode(QLineEdit.Normal)   # oddiy
lineedit.setEchoMode(QLineEdit.NoEcho)   # hech narsa ko'rsatmaydi

# Cheklash
lineedit.setMaxLength(20)                # max 20 ta belgi
lineedit.setReadOnly(True)               # faqat o'qish

# Validator (faqat son kiritish)
from PyQt5.QtGui import QIntValidator, QDoubleValidator, QRegExpValidator
lineedit.setValidator(QIntValidator(0, 100)) # 0-100 oralig'ida son
lineedit.setValidator(QDoubleValidator())     # kasr son

# Tanlash
lineedit.selectAll()                     # hammasini tanlash
lineedit.selectedText()                  # tanlangan matnni olish
```

QLineEdit signallari:

```
lineedit.textChanged.connect(...) # har harf kiritilganda
lineedit.textEdited.connect(...)  # foydalanuvchi o'zgartirsa
lineedit.returnPressed.connect(...) # Enter bosilganda
lineedit.editingFinished.connect(...) # tahrirlash tugaganda
lineedit.cursorPositionChanged.connect(...) # kursor o'zgarganda
```

4. Shrift (Font) O'rnatish

```
from PyQt5.QtGui import QFont

# Shrift yaratish
shrft = QFont()
shrft.setFamily("Arial") # shrift turi
shrft.setPointSize(14)   # o'lcham (pt)
shrft.setBold(True)      # qalin
shrft.setItalic(True)    # kursiv
shrft.setUnderline(True) # tagiga chiziq

# QLabel ga o'rnatish
label.setFont(shrft)

# QLineEdit ga o'rnatish
```

```
lineEdit.setFont(shrift)
```

Qisqa usul

```
label.setFont(QFont("Times New Roman", 16, QFont.Bold))
```

5. setStyleSheet — CSS bilan Bezash

setStyleSheet — bu Qt ning CSS ga o'xshash tizimi. Deyarli barcha CSS xususiyatlari ishlaydi!

QLabel uchun:

```
label.setStyleSheet("""
    QLabel {
        color: white;
        background-color: #2c3e50;
        font-size: 18px;
        font-weight: bold;
        font-family: Arial;
        border: 2px solid #3498db;
        border-radius: 8px;
        padding: 10px;
    }
""")
```

QLineEdit uchun:

```
lineEdit.setStyleSheet("""
    QLineEdit {
        color: #2c3e50;
        background-color: #ecf0f1;
        font-size: 14px;
        border: 2px solid #bdc3c7;
        border-radius: 6px;
        padding: 8px;
    }
    QLineEdit:focus {
        border: 2px solid #3498db;
        background-color: white;
    }
    QLineEdit:hover {
        border: 2px solid #95a5a6;
    }
    QLineEdit:disabled {
        background-color: #d5dbdb;
        color: #808b96;
    }
""")
```

 `:focus`, `:hover`, `:disabled` — bular **pseudo-state** lar, CSS dagi kabi ishlaydi!

6. Taqqoslash Jadvali

Xususiyat	QLabel	QLineEdit
Vazifasi	Matn/rasm ko'rsatish	Matn kiritish
Foydalanuvchi tahrirlashi	✗	✓
Matn olish <code>.text()</code>	✓	✓
Matn o'rnatish <code>.setText()</code>	✓	✓
Placeholder	✗	✓
Parol rejimi	✗	✓
Validator	✗	✓
Rasm ko'rsatish	✓	✗
HTML qo'llab-quvvatlash	✓	✗
Signallar soni	2 ta	5+ ta
WordWrap	✓	✗

7. To'liq Amaliy Misol — Ro'yxatdan o'tish formasi

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QFont, QPixmap, QIntValidator
from PyQt5.QtCore import Qt

class RoyxatdanOtish(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Ro'yxatdan o'tish")
        self.setFixedSize(400, 500)
        self.setStyleSheet("background-color: #f5f6fa;")
        self.interfeysni_yaratish()

    def interfeysni_yaratish(self):
        tartib = QVBoxLayout(self)
        tartib.setContentsMargins(40, 30, 40, 30)
        tartib.setSpacing(12)

        # === SARLAVHA (QLabel) ===
        sarlavha = QLabel("Ro'yxatdan o'tish")
        sarlavha.setAlignment(Qt.AlignCenter)
        sarlavha.setStyleSheet("""
            QLabel {
                font-size: 24px;
                font-weight: bold;
                color: #2c3e50;
                padding: 10px;
                background-color: #3498db;
                color: white;
                border-radius: 10px;
            }
        """)
```

```

    """)
    tartib.addWidget(sarlavha)

    # === ISM ===
    ism_label = QLabel("👤 Ism:") # QLabel – faqat ko'rsatish
    ism_label.setStyleSheet("font-size: 14px; color: #555; font-weight: bold;")
    self.ism_input = QLineEdit() # QLineEdit – kiritish uchun
    self.ism_input.setPlaceholderText("Ismingizni kiriting...")
    self.ism_input.setStyleSheet(self.input_stili())
    self.ism_input.setMinimumHeight(40)
    tartib.addWidget(ism_label)
    tartib.addWidget(self.ism_input)

    # === EMAIL ===
    email_label = QLabel("✉ Email:")
    email_label.setStyleSheet("font-size: 14px; color: #555; font-weight:
bold;")
    self.email_input = QLineEdit()
    self.email_input.setPlaceholderText("email@example.com")
    self.email_input.setStyleSheet(self.input_stili())
    self.email_input.setMinimumHeight(40)
    tartib.addWidget(email_label)
    tartib.addWidget(self.email_input)

    # === YOSH (Validator bilan) ===
    yosh_label = QLabel("🎂 Yosh (1-120):")
    yosh_label.setStyleSheet("font-size: 14px; color: #555; font-weight: bold;")
    self.yosh_input = QLineEdit()
    self.yosh_input.setPlaceholderText("Yoshingizni kiriting...")
    self.yosh_input.setValidator(QIntValidator(1, 120)) # faqat son!
    self.yosh_input.setStyleSheet(self.input_stili())
    self.yosh_input.setMinimumHeight(40)
    tartib.addWidget(yosh_label)
    tartib.addWidget(self.yosh_input)

    # === PAROL ===
    parol_label = QLabel("🔒 Parol:")
    parol_label.setStyleSheet("font-size: 14px; color: #555; font-weight:
bold;")
    self.parol_input = QLineEdit()
    self.parol_input.setPlaceholderText("Parol kiriting...")
    self.parol_input.setEchoMode(QLineEdit.Password) # *** ko'rsatadi
    self.parol_input.setMaxLength(20)
    self.parol_input.setStyleSheet(self.input_stili())
    self.parol_input.setMinimumHeight(40)
    tartib.addWidget(parol_label)
    tartib.addWidget(self.parol_input)

    # === NATIJA LABEL (dinamik o'zgaradi) ===
    self.natija_label = QLabel("")
    self.natija_label.setAlignment(Qt.AlignCenter)
    self.natija_label.setWordWrap(True)
    self.natija_label.setMinimumHeight(40)
    tartib.addWidget(self.natija_label)

    # === TUGMA ===
    tugma = QPushButton("✅ Ro'yxatdan o'tish")

```

```

tugma.setMinimumHeight(45)
tugma.setStyleSheet("""
    QPushButton {
        background-color: #27ae60;
        color: white;
        font-size: 15px;
        font-weight: bold;
        border-radius: 8px;
        border: none;
    }
    QPushButton:hover {
        background-color: #2ecc71;
    }
    QPushButton:pressed {
        background-color: #1e8449;
    }
""")
tugma.clicked.connect(self.formni_tekshir)
tartib.addWidget(tugma)

# Real vaqtda tekshirish
self.ism_input.textChanged.connect(self.real_vaqtda_tekshir)
self.email_input.textChanged.connect(self.real_vaqtda_tekshir)

def input_stili(self):
    return """
        QLineEdit {
            font-size: 14px;
            padding: 8px 12px;
            border: 2px solid #ddd;
            border-radius: 8px;
            background-color: white;
            color: #2c3e50;
        }
        QLineEdit:focus {
            border: 2px solid #3498db;
            background-color: #eaf4fb;
        }
        QLineEdit:hover {
            border: 2px solid #aab7b8;
        }
    """

def real_vaqtda_tekshir(self):
    """Har harf kiritilganda tekshiradi"""
    if len(self.ism_input.text()) > 0:
        self.natija_label.setText("👉 Yozilmoqda...")
        self.natija_label.setStyleSheet("color: #f39c12; font-size: 13px;")

def formni_tekshir(self):
    ism = self.ism_input.text().strip()
    email = self.email_input.text().strip()
    yosh = self.yosh_input.text().strip()
    parol = self.parol_input.text()

    # Tekshirish
    if not ism:

```

```

        self.xato_korsot("❌ Ism bo'sh bo'lmasin!")
        return
    if "@" not in email:
        self.xato_korsot("❌ Email noto'g'ri!")
        return
    if not yosh:
        self.xato_korsot("❌ Yoshni kiriting!")
        return
    if len(parol) < 6:
        self.xato_korsot("❌ Parol kamida 6 ta belgi!")
        return

    # Muvaffaqiyat
    self.natija_label.setText(
        f"✅ Xush kelibsiz, {ism}!\nEmail: {email} | Yosh: {yosh}"
    )
    self.natija_label.setStyleSheet("""
        color: white;
        background-color: #27ae60;
        border-radius: 8px;
        padding: 8px;
        font-size: 13px;
        font-weight: bold;
    """)

def xato_korsot(self, xabar):
    self.natija_label.setText(xabar)
    self.natija_label.setStyleSheet("""
        color: white;
        background-color: #e74c3c;
        border-radius: 8px;
        padding: 8px;
        font-size: 13px;
        font-weight: bold;
    """)

app = QApplication(sys.argv)
oyna = RoyxatdanOtish()
oyna.show()
sys.exit(app.exec_())

```

8. Xulosa

QLabel → Ko'rsatish uchun | .setText() | rasm, HTML, matn
 QLineEdit → Kiritish uchun | .text() | parol, validator, placeholder

Ikkalasiga ham:

- ✅ setFont() — shrift
- ✅ setFixedSize() — o'lcham
- ✅ setStyleSheet() — CSS bilan bezash

💡 **Eslab qol:** QLabel — doskadagi yozuv, QLineEdit — daftardagi bo'sh qator!

● SAVOL 3:

QMESSAGEBOX **VIDJETINING VAZIFASI VA TURLARI** (**INFORMATION** , **WARNING** , **CRITICAL** , **QUESTION**). **STATIK FUNKSIYALAR VA PIKTOGRAMMALARDAN FOYDALANISH USULLARINI BATAFSIL TUSHUNTIRING.**

1. QMessageBox nima?

QMessageBox — foydalanuvchiga **qisqa xabar ko'rsatish** yoki **qaror qabul qildirish** uchun ishlatiladigan tayyor dialog oynasi.

Dastur → "Faylni o'chirasizmi?" → Foydalanuvchi: Ha / Yo'q
Dastur → "Xato yuz berdi!" → Foydalanuvchi: OK
Dastur → "Saqlandi  " → Foydalanuvchi: OK

2. Ishlatishning 2 usuli

1. Statik funksiyalar → Tez, 1 qator kod (ko'p ishlatiladi)
 2. Ob'ekt yaratish → Moslashuvchan, kengaytirish mumkin
-

3. Statik Funksiyalar — Tez Ishlatish

information — Ma'lumot berish

```
from PyQt5.QtWidgets import QMessageBox
```

```
QMessageBox.information(  
    self,                                # ota widget  
    "Sarlavha",                          # oyna sarlavhasi  
    "Fayl muvaffaqiyatli saqlandi!",     # xabar  
    QMessageBox.Ok                       # tugmalar (ixtiyoriy)  
)
```

 Ko'k "i" belgisi — oddiy ma'lumot

warning — Ogohlantirish

```
QMessageBox.warning(  
    self,  
    "Ogohlantirish",  
    "Fayl o'zgartirilgan! Saqlamasdan chiqmoqchimisiz?",  
    QMessageBox.Yes | QMessageBox.No     # ikki tugma  
)
```

 Sariq uchburchak — ehtiyot bo'lish kerak

✖ critical — Jiddiy xato

```
QMessageBox.critical(  
    self,  
    "Xato",  
    "Ma'lumotlar bazasiga ulanib bo'lmadi!",  
    QMessageBox.Ok  
)
```

● Qizil X — jiddiy muammo

? question — Savol berish

```
javob = QMessageBox.question(  
    self,  
    "Tasdiqlash",  
    "Haqiqatan ham o'chirmoqchimisiz?",  
    QMessageBox.Yes | QMessageBox.No,  
    QMessageBox.No # default tugma  
)
```

```
if javob == QMessageBox.Yes:  
    print("O'chirildi!")  
else:  
    print("Bekor qilindi!")
```

● So'roq belgisi — foydalanuvchidan javob so'rash

4. Barcha Tugma Turlari

QMessageBox.Ok	# OK
QMessageBox.Cancel	# Bekor qilish
QMessageBox.Yes	# Ha
QMessageBox.No	# Yo'q
QMessageBox.Save	# Saqlash
QMessageBox.Discard	# Saqlamaslik
QMessageBox.Apply	# Qo'llash
QMessageBox.Close	# Yopish
QMessageBox.Abort	# To'xtatish
QMessageBox.Retry	# Qayta urinish
QMessageBox.Ignore	# E'tiborsiz qoldirish
QMessageBox.Help	# Yordam
QMessageBox.Reset	# Qayta o'rnatish

Bir vaqtda bir nechta:

```
QMessageBox.Yes | QMessageBox.No | QMessageBox.Cancel
```

5. Qaytarish Qiymatlari

```
javob = QMessageBox.question(  
    self, "Savol", "Davom etasizmi?",  
    QMessageBox.Yes | QMessageBox.No | QMessageBox.Cancel
```

```
)
```

```
if javob == QMessageBox.Yes:
    print("Ha dedi")
elif javob == QMessageBox.No:
    print("Yo'q dedi")
elif javob == QMessageBox.Cancel:
    print("Bekor qildi")
```

6. Ob'ekt Yaratib Ishlatish — Moslashuvchan Usul

```
from PyQt5.QtWidgets import QMessageBox
from PyQt5.QtGui import QIcon, QPixmap
from PyQt5.QtCore import Qt

xabar = QMessageBox()

# Sarlavha va matn
xabar.setWindowTitle("Muhim Xabar")
xabar.setText("Asosiy xabar matni") # katta matn
xabar.setInformativeText("Qo'shimcha tushuntirish...") # kichik matn
xabar.setDetailedText("Log: xato 404...\nFile: main.py") # "Batafsil" tugmasi

# Ikon turi
xabar.setIcon(QMessageBox.Information) # yoki Warning, Critical, Question

# Tugmalar
xabar.setStandardButtons(QMessageBox.Yes | QMessageBox.No | QMessageBox.Cancel)
xabar.setDefaultButton(QMessageBox.Yes) # Enter bosilganda
xabar.setEscapeButton(QMessageBox.Cancel) # Esc bosilganda

# Ko'rsatish va javob olish
natija = xabar.exec_()

if natija == QMessageBox.Yes:
    print("Ha!")
```

7. Piktogrammalar (Iconlar)

Standart ikonalar:

Ikon o'rnatish usullari

1. Standart Qt ikonalari

xabar.setIcon(QMessageBox.NoIcon)	# ikonsiz
xabar.setIcon(QMessageBox.Information)	#  ma'lumot
xabar.setIcon(QMessageBox.Warning)	#  ogohlantirish
xabar.setIcon(QMessageBox.Critical)	#  xato
xabar.setIcon(QMessageBox.Question)	#  savol

2. O'z rasmingizni ishlatish

```
xabar.setIconPixmap(QPixmap("logo.png").scaled(64, 64))
```

```
# 3. Oyna ikonasi (sarlavha barida)
xabar.setWindowIcon(QIcon("icon.png"))
```

Standart OS ikonalaridan foydalanish:

```
from PyQt5.QtWidgets import QStyle, QApplication
```

```
# Tizim ikonalarini olish
stil = QApplication.style()
```

```
ikon = stil.standardIcon(QStyle.SP_MessageBoxInformation)
ikon = stil.standardIcon(QStyle.SP_MessageBoxWarning)
ikon = stil.standardIcon(QStyle.SP_MessageBoxCritical)
ikon = stil.standardIcon(QStyle.SP_MessageBoxQuestion)
```

```
xabar.setWindowIcon(ikon)
```

8. StyleSheet bilan Bezash

```
xabar = QMessageBox()
xabar.setWindowTitle("Styled Xabar")
xabar.setText("Bu bezatilgan xabar!")
xabar.setIcon(QMessageBox.Information)
```

```
xabar.setStyleSheet("""
    QMessageBox {
        background-color: #2c3e50;
        color: white;
    }
    QMessageBox QLabel {
        color: white;
        font-size: 14px;
        min-width: 250px;
    }
    QPushButton {
        background-color: #3498db;
        color: white;
        border-radius: 6px;
        padding: 6px 20px;
        font-size: 13px;
        min-width: 80px;
    }
    QPushButton:hover {
        background-color: #2980b9;
    }
    QPushButton:pressed {
        background-color: #1a6fa8;
    }
""")
```

```
xabar.exec_()
```

9. To'liq Amaliy Misol

```

import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QIcon
from PyQt5.QtCore import Qt

class AsosiyOyna(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("QMessageBox Namunalar")
        self.setFixedSize(420, 380)
        self.setStyleSheet("background-color: #f0f2f5;")
        self.interfeysni_yaratish()

    def interfeysni_yaratish(self):
        markaziy = QWidget()
        self.setCentralWidget(markaziy)
        tartib = QVBoxLayout(markaziy)
        tartib.setContentsMargins(40, 30, 40, 30)
        tartib.setSpacing(14)

        sarlavha = QLabel("QMessageBox Turlari")
        sarlavha.setAlignment(Qt.AlignCenter)
        sarlavha.setStyleSheet("""
            font-size: 20px; font-weight: bold;
            color: white; background-color: #2c3e50;
            border-radius: 10px; padding: 12px;
        """)
        tartib.addWidget(sarlavha)

        tugmalar = [
            ("● Information", "#2980b9", self.info_korsot),
            ("● Warning", "#f39c12", self.warning_korsot),
            ("● Critical", "#e74c3c", self.critical_korsot),
            ("? Question", "#8e44ad", self.question_korsot),
            ("⚙ Moslashtirilgan", "#27ae60", self.custom_korsot),
        ]

        for matn, rang, funksiya in tugmalar:
            tugma = QPushButton(matn)
            tugma.setMinimumHeight(42)
            tugma.setStyleSheet(f"""
                QPushButton {{
                    background-color: {rang};
                    color: white;
                    font-size: 14px;
                    font-weight: bold;
                    border-radius: 8px;
                    border: none;
                }}
                QPushButton: hover {{ opacity: 0.85; }}
                QPushButton: pressed {{ padding-top: 2px; }}
            """)
            tugma.clicked.connect(funksiya)
            tartib.addWidget(tugma)

```

```

def info_korsot(self):
    QMessageBox.information(
        self,
        "Ma'lumot",
        "✅ Fayl muvaffaqiyatli saqlandi!\nYo'l: /home/user/hujjat.txt",
        QMessageBox.Ok
    )

def warning_korsot(self):
    javob = QMessageBox.warning(
        self,
        "Ogohlantirish",
        "⚠ Faylda saqlanmagan o'zgarishlar bor.\n"
        "Saqlamasdan chiqmoqchimisiz?",
        QMessageBox.Save | QMessageBox.Discard | QMessageBox.Cancel,
        QMessageBox.Save # default
    )
    if javob == QMessageBox.Save:
        self.natija_label_yangilar("💾 Saqlandi va chiqildi")
    elif javob == QMessageBox.Discard:
        self.natija_label_yangilar("🗑 Saqlanmadi, chiqildi")
    else:
        self.natija_label_yangilar("🔄 Bekor qilindi")

def critical_korsot(self):
    xabar = QMessageBox(self)
    xabar.setWindowTitle("Kritik Xato")
    xabar.setIcon(QMessageBox.Critical)
    xabar.setText("❌ Ulanishda xatolik yuz berdi!")
    xabar.setInformativeText(
        "Ma'lumotlar bazasiga ulanib bo'lmadi."
    )
    xabar.setDetailedText(
        "Xato: Connection refused\n"
        "Host: localhost:5432\n"
        "Vaqt: 2024-01-15 14:30:22\n"
        "Qayta urinib ko'ring yoki administratorga murojaat qiling."
    )
    xabar.setStandardButtons(QMessageBox.Retry | QMessageBox.Abort)
    xabar.setDefaultButton(QMessageBox.Retry)

    natija = xabar.exec_()
    if natija == QMessageBox.Retry:
        self.natija_label_yangilar("🔄 Qayta urinilmoqda...")
    else:
        self.natija_label_yangilar("🛑 To'xtatildi")

def question_korsot(self):
    javob = QMessageBox.question(
        self,
        "Tasdiqlash",
        "🗑 Tanlangan 5 ta faylni o'chirmoqchimisiz?\n"
        "Bu amalni qaytarib bo'lmaydi!",
        QMessageBox.Yes | QMessageBox.No,
        QMessageBox.No # xavfsiz default
    )

```

```

if javob == QMessageBox.Yes:
    self.natija_label_yangilar("🗑️ Fayllar o'chirildi!")
else:
    self.natija_label_yangilar("👉 O'chirish bekor qilindi")

def custom_korsot(self):
    """Ob'ekt yaratib, to'liq moslashtirilgan"""
    xabar = QMessageBox(self)
    xabar.setWindowTitle("Yangilanish mavjud")
    xabar.setText("🚀 Yangi versiya mavjud: v2.0.1")
    xabar.setInformativeText(
        "Yangilanishni hozir o'rnatmoqchimisiz?"
    )
    xabar.setDetailedText(
        "Yangiliklar:\n"
        "- Tezlik 40% oshirildi\n"
        "- 12 ta xato tuzatildi\n"
        "- Yangi dark mode qo'shildi"
    )
    xabar.setIcon(QMessageBox.Information)
    xabar.setStandardButtons(
        QMessageBox.Yes | QMessageBox.No | QMessageBox.Cancel
    )
    xabar.setDefaultButton(QMessageBox.Yes)
    xabar.setEscapeButton(QMessageBox.Cancel)

    xabar.setStyleSheet("""
        QMessageBox {
            background-color: #1e272e;
        }
        QMessageBox QLabel {
            color: #dfe6e9;
            font-size: 13px;
            min-width: 280px;
        }
        QPushButton {
            background-color: #0984e3;
            color: white;
            border-radius: 6px;
            padding: 6px 18px;
            font-weight: bold;
            min-width: 75px;
        }
        QPushButton:hover {
            background-color: #74b9ff;
        }
    """)

    natija = xabar.exec_()
    if natija == QMessageBox.Yes:
        self.natija_label_yangilar("⬇️ Yangilanish boshlandi...")
    elif natija == QMessageBox.No:
        self.natija_label_yangilar("⏮️ Keyinroq eslatiladi")
    else:
        self.natija_label_yangilar("❌ Bekor qilindi")

def natija_label_yangilar(self, matn):

```

```
QMessageBox.information(self, "Natija", matn)
```

```
app = QApplication(sys.argv)
oyna = AsosiyOyna()
oyna.show()
sys.exit(app.exec_())
```

10. Xulosa Jadvali

Tur	Statik funksiya	Ikona	Qachon ishlatish
Information	<code>QMessageBox.information()</code>		Muvaffaqiyat, ma'lumot
Warning	<code>QMessageBox.warning()</code>		Ehtiyot bo'lish kerak
Critical	<code>QMessageBox.critical()</code>		Jiddiy xato
Question	<code>QMessageBox.question()</code>		Foydalanuvchidan qaror

Statik funksiya → 1 qator, tez, sodda

Ob'ekt yaratish → `setDetailedText`, `StyleSheet`, maxsus tugmalar kerak bo'lganda

 **Eslab qol:** `question()` dan qaytgan qiymatni **har doim** tekshir — foydalanuvchi `Yes` yoki `No` bosganini bilish shart!

● SAVOL 4:

PYQT5 DA MENYU YARATISH JARAYONI: `QMenuBar`, `QMenu`, `QAction` OB'EKTLARI VA ULAR O'RTASIDAGI MUNOSABAT. KEYBOARD SHORTCUT VA IKONKALAR QO'SHISH USULLARINI KO'RSATING.

1. Menyü Tizimi — Asosiy Tushuncha

`QMainWindow` (oyna)



`QMenuBar` (menyu paneli: Fayl, Tahrirlash, Yordamasi...)



`QMenu` (ichki menyular: Yangi, Ochish, Saqlash...)



`QAction` (harakat: Fayl ochish, Matni kesish...)

Analogy:

Restorandagi menyu

```
|— Taomlar bo'limi (QMenuBar)
|   |— Birinchi taom (QMenu)
|       |— Shorba (QAction)
```

```
| | | Salatasi (QAction)
| | | ...
| | | Ikkinchi taom (QMenu)
| | | ...
```

2. QAction — Asosiy Birlik

QAction — bu bir ta harakat (action) ni ifodalaydi. Tugma, menyu, toolbar da ishlatilishi mumkin.

```
from PyQt5.QtWidgets import QMainWindow, QApplication
from PyQt5.QtGui import QIcon
from PyQt5.QtCore import Qt

# QAction yaratish
ochish_action = QAction("Ochish", self)
ochish_action.setShortcut("Ctrl+O")           # Keyboard shortcut
ochish_action.setStatusTip("Faylni ochish")    # Status barida ko'rsatiladi
ochish_action.triggered.connect(self.fayl_ochish) # Signal-Slot

# Ikonka qo'shish
ochish_action.setIcon(QIcon("icons/open.png"))

# Enabled/Disabled qilish
ochish_action.setEnabled(True)
```

3. QMenuBar — Menyu Paneli

```
from PyQt5.QtWidgets import QMainWindow, QApplication

class AsosiyOyna(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Menyu Namunasi")
        self.setGeometry(100, 100, 600, 400)

        # QMenuBar olish (QMainWindow da avtomatik mavjud)
        menubar = self.menuBar()

        # Menyu qo'shish
        fayl_menyu = menubar.addMenu("Fayl")
        tahrirlash_menyu = menubar.addMenu("Tahrirlash")
        yordamasi_menyu = menubar.addMenu("Yordamasi")
```

4. QMenu — Ichki Menyular

```
class AsosiyOyna(QMainWindow):
    def __init__(self):
        super().__init__()

        menubar = self.menuBar()
```



```

# === FAYL MENYU ===
fayl_menyu = menubar.addMenu("📁 Fayl")

# Harakat yaratish
yangi_action = QAction("Yangi", self)
yangi_action.setShortcut("Ctrl+N")
yangi_action.triggered.connect(self.yangi_fayl)

ochish_action = QAction("Ochish", self)
ochish_action.setShortcut("Ctrl+O")
ochish_action.triggered.connect(self.fayl_ochish)

saqlash_action = QAction("Saqlash", self)
saqlash_action.setShortcut("Ctrl+S")
saqlash_action.triggered.connect(self.fayl_saqlash)

chiqish_action = QAction("Chiqish", self)
chiqish_action.setShortcut("Ctrl+Q")
chiqish_action.triggered.connect(self.close)

# Menyuga harakat qo'shish
fayl_menyu.addAction(yangi_action)
fayl_menyu.addAction(ochish_action)
fayl_menyu.addSeparator() # — chiziq
fayl_menyu.addAction(saqlash_action)
fayl_menyu.addSeparator()
fayl_menyu.addAction(chiqish_action)

# === TAHRIRLASH MENYU ===
tahrirlash_menyu = menubar.addMenu("✎ Tahrirlash")

kesish_action = QAction("Kesish", self)
kesish_action.setShortcut("Ctrl+X")
tahrirlash_menyu.addAction(kesish_action)

nusxa_action = QAction("Nusxa", self)
nusxa_action.setShortcut("Ctrl+C")
tahrirlash_menyu.addAction(nusxa_action)

yapistir_action = QAction("Yapistir", self)
yapistir_action.setShortcut("Ctrl+V")
tahrirlash_menyu.addAction(yapistir_action)

def yangi_fayl(self):
    print("Yangi fayl yaratildi!")

def fayl_ochish(self):
    print("Fayl oynasi ochildi!")

def fayl_saqlash(self):
    print("Fayl saqlandi!")

```

5. Keyboard Shortcut — Qisqa Tugmalar

Ishlatiladigan kombinatsiyalar:

```
"Ctrl+N"      # Windows/Linux: Ctrl+N, macOS: Cmd+N
"Shift+Ctrl+S" # Shift + Ctrl + S
"Alt+F"       # Alt + F
"F5"          # Funktsiya tugmasi
"Delete"      # Delete tugmasi
"Return"      # Enter tugmasi
"Space"       # Boshshalik
"Escape"      # Esc tugmasi
```

Platform-shunosli (cross-platform) shortcut:

```
from PyQt5.QtGui import QKeySequence

# Avtomatik tanlaydi: Cmd+N macOS da, Ctrl+N boshqalarda
saqlash_action.setShortcut(QKeySequence.Save)      # Ctrl+S
kopiylash_action.setShortcut(QKeySequence.Copy)    # Ctrl+C
kesish_action.setShortcut(QKeySequence.Cut)        # Ctrl+X
yapistirish_action.setShortcut(QKeySequence.Paste) # Ctrl+V
izini_qaytarish_action.setShortcut(QKeySequence.Undo) # Ctrl+Z
izini_qoqlash_action.setShortcut(QKeySequence.Redo) # Ctrl+Y
```

6. Ikonkalar Qo'shish

Local fayldan:

```
from PyQt5.QtGui import QIcon

saqlash_action = QAction("Saqlash", self)
saqlash_action.setIcon(QIcon("icons/save.png")) # 32x32 px
```

Tizim ikonalaridan (OS):

```
from PyQt5.QtWidgets import QStyle, QApplication

stil = QApplication.style()

ochish_ikon = stil.standardIcon(QStyle.SP_DirOpenIcon)
qo_shish_ikon = stil.standardIcon(QStyle.SP_FileIcon)
ochirib_tashlash_ikon = stil.standardIcon(QStyle.SP_DialogNoIcon)
saqlash_ikon = stil.standardIcon(QStyle.SP_DialogYesButton)

saqlash_action.setIcon(saqlash_ikon)
```

SVG (Vektorli):

```
ikon = QIcon()
ikon.addPixmap(QPixmap("icons/save.svg"), QIcon.Normal)
saqlash_action.setIcon(ikon)
```

7. Submenu — Ichki Menyular

```

fayl_menyu = menubar.addAction("Fayl")

# Submenu yaratish
ochish_submenu = fayl_menyu.addAction("Ochish") # ← new!

# Submenuya harakat qo'shish
oxirgi_fayllar = []
for fayl in ["document1.txt", "document2.txt"]:
    action = QAction(fayl, self)
    action.triggered.connect(lambda checked, f=fayl: self.ochish(f))
    ochish_submenu.addAction(action)

```

```

Fayl
├─ Yangi
├─ Ochish →
│   ├── document1.txt
│   ├── document2.txt
│   └── document3.txt
├─ Saqlash
└─ Chiqish

```

8. Checkbox va Radio Button Harakatlari

```

# Checkbox action (On/Off)
show_toolbar_action = QAction("Toolbar ko'rsatish", self, checkable=True)
show_toolbar_action.setChecked(True)
show_toolbar_action.triggered.connect(self.toolbar_toggle)

menyu.addAction(show_toolbar_action)

# Radio button (faqat 1 ta tanlangan)
from PyQt5.QtWidgets import QActionGroup

group = QActionGroup(self)

svetliy_action = QAction("Svetliy tema", self, checkable=True)
svetliy_action.setChecked(True)
group.addAction(svetliy_action)

qorong_action = QAction("Qorong' tema", self, checkable=True)
group.addAction(qorong_action)

tema_menyu = menubar.addAction("Tema")
tema_menyu.addAction(group.actions())

group.triggered.connect(self.tema_ozgartir)

```

9. To'liq Amaliy Misol

```

import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QIcon, QKeySequence, QPixmap
from PyQt5.QtCore import Qt

```

```

class NotePadApp(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("NotePad - Menyu Namunasi")
        self.setGeometry(200, 100, 700, 500)
        self.setStyleSheet("background-color: #f5f5f5;")

        # Markaziy widget
        self.text_edit = QPlainTextEdit()
        self.text_edit.setPlainText("Matnni shu yerga yozing...")
        self.setCentralWidget(self.text_edit)

        # Status bar
        self.statusBar().showMessage("Tayyor", 5000)

        # Menyu yaratish
        self.menyu_yaratish()

    def menyu_yaratish(self):
        menubar = self.menuBar()

        # ===== FAYL MENYU =====
        fayl_menyu = menubar.addMenu("📁 Fayl")

        # Yangi
        yangi_action = QAction("Yangi", self)
        yangi_action.setShortcut(QKeySequence.New) # Ctrl+N
        yangi_action.setStatusTip("Yangi fayl yaratish")
        yangi_action.setIcon(self.get_icon(QStyle.SP_FileIcon))
        yangi_action.triggered.connect(self.yangi_fayl)
        fayl_menyu.addAction(yangi_action)

        # Ochish
        ochish_action = QAction("Ochish", self)
        ochish_action.setShortcut(QKeySequence.Open) # Ctrl+O
        ochish_action.setStatusTip("Faylni ochish")
        ochish_action.setIcon(self.get_icon(QStyle.SP_DirOpenIcon))
        ochish_action.triggered.connect(self.fayl_ochish)
        fayl_menyu.addAction(ochish_action)

        # Saqlash
        saqlash_action = QAction("Saqlash", self)
        saqlash_action.setShortcut(QKeySequence.Save) # Ctrl+S
        saqlash_action.setStatusTip("Faylni saqlash")
        saqlash_action.setIcon(self.get_icon(QStyle.SP_DriveFDIcon))
        saqlash_action.triggered.connect(self.fayl_saqlash)
        fayl_menyu.addAction(saqlash_action)

        # Saqlash sifatida
        saqlash_sifatida_action = QAction("Saqlash sifatida...", self)
        saqlash_sifatida_action.setShortcut("Ctrl+Shift+S")
        saqlash_sifatida_action.triggered.connect(self.fayl_saqlash_sifatida)
        fayl_menyu.addAction(saqlash_sifatida_action)

        # Ajratuvchi chiziq

```

```

fayl_menyu.addSeparator()

# Oxirgi fayllar (submenu)
oxirgi_menyu = fayl_menyu.addMenu("Oxirgi fayllar")
for i in range(1, 4):
    action = QAction(f"document{i}.txt", self)
    action.triggered.connect(lambda checked, n=i: self.oxirgi_ochish(n))
    oxirgi_menyu.addAction(action)

# Ajratuvchi
fayl_menyu.addSeparator()

# Chiqish
chiqish_action = QAction("Chiqish", self)
chiqish_action.setShortcut(QKeySequence.Quit) # Ctrl+Q
chiqish_action.setStatusTip("Dasturdan chiqish")
chiqish_action.triggered.connect(self.close)
fayl_menyu.addAction(chiqish_action)

# ===== TAHRIRLASH MENYU =====
tahrirlash_menyu = menubar.addMenu("✎ Tahrirlash")

# Qaytarish
qaytarish_action = QAction("Qaytarish", self)
qaytarish_action.setShortcut(QKeySequence.Undo) # Ctrl+Z
qaytarish_action.triggered.connect(self.text_edit.undo)
tahrirlash_menyu.addAction(qaytarish_action)

# Qayta qilish
qayta_action = QAction("Qayta qilish", self)
qayta_action.setShortcut(QKeySequence.Redo) # Ctrl+Y
qayta_action.triggered.connect(self.text_edit.redo)
tahrirlash_menyu.addAction(qayta_action)

tahrirlash_menyu.addSeparator()

# Kesish
kesish_action = QAction("Kesish", self)
kesish_action.setShortcut(QKeySequence.Cut) # Ctrl+X
kesish_action.triggered.connect(self.text_edit.cut)
tahrirlash_menyu.addAction(kesish_action)

# Nusxa
nusxa_action = QAction("Nusxa", self)
nusxa_action.setShortcut(QKeySequence.Copy) # Ctrl+C
nusxa_action.triggered.connect(self.text_edit.copy)
tahrirlash_menyu.addAction(nusxa_action)

# Yapistir
yapistir_action = QAction("Yapistir", self)
yapistir_action.setShortcut(QKeySequence.Paste) # Ctrl+V
yapistir_action.triggered.connect(self.text_edit.paste)
tahrirlash_menyu.addAction(yapistir_action)

tahrirlash_menyu.addSeparator()

# Hammasini tanlash

```

```

tanlash_action = QAction("Hammasini tanlash", self)
tanlash_action.setShortcut(QKeySequence.SelectAll) # Ctrl+A
tanlash_action.triggered.connect(self.text_edit.selectAll)
tahrirlash_menyu.addAction(tanlash_action)

# ===== KO'RINISH MENYU =====
korinish_menyu = menubar.addMenu("👁️ Ko'rinish")

# Toolbar (checkbox)
toolbar_action = QAction("Toolbar", self, checkable=True)
toolbar_action.setChecked(True)
toolbar_action.triggered.connect(self.toolbar_toggle)
korinish_menyu.addAction(toolbar_action)

# Status bar (checkbox)
statusbar_action = QAction("Status bar", self, checkable=True)
statusbar_action.setChecked(True)
statusbar_action.triggered.connect(self.statusbar_toggle)
korinish_menyu.addAction(statusbar_action)

korinish_menyu.addSeparator()

# Tema (radio buttons)
tema_group = QActionGroup(self)

svetliy_action = QAction("Svetliy tema", self, checkable=True)
svetliy_action.setChecked(True)
tema_group.addAction(svetliy_action)

qorong_action = QAction("Qorong' tema", self, checkable=True)
tema_group.addAction(qorong_action)

korinish_menyu.addAction(tema_group.actions())
tema_group.triggered.connect(self.tema_ozgartir)

# ===== YORDAMASI MENYU =====
yordamasi_menyu = menubar.addMenu("❓ Yordamasi")

haqida_action = QAction("Dastur haqida", self)
haqida_action.triggered.connect(self.haqida_korsot)
yordamasi_menyu.addAction(haqida_action)

# ===== TOOLBAR =====
self.toolbar = self.addToolBar("Asboblari")
self.toolbar.addAction(yangi_action)
self.toolbar.addAction(ochish_action)
self.toolbar.addAction(saqlash_action)
self.toolbar.addSeparator()
self.toolbar.addAction(kesish_action)
self.toolbar.addAction(nusxa_action)
self.toolbar.addAction(yapistir_action)

def get_icon(self, style_icon):
    """OS ikonalarini olish"""
    stil = QApplication.style()
    return stil.standardIcon(style_icon)

```

```
def yangi_fayl(self):
    self.text_edit.clear()
    self.statusBar().showMessage("Yangi fayl yaratildi", 2000)

def fayl_ochish(self):
    fayl_nomi, _ = QFileDialog.getOpenFileName(
        self, "Faylni ochish", "", "Text fayllar (*.txt);;Barcha fayllar (*)"
    )
    if fayl_nomi:
        with open(fayl_nomi, 'r', encoding='utf-8') as f:
            self.text_edit.setText(f.read())
            self.statusBar().showMessage(f"Ochildi: {fayl_nomi}", 2000)

def fayl_saqlash(self):
    self.statusBar().showMessage("Fayl saqlandi ✓", 2000)

def fayl_saqlash_sifatida(self):
    fayl_nomi, _ = QFileDialog.getSaveFileName(
        self, "Saqlash sifatida", "", "Text fayllar (*.txt);;Barcha fayllar (*)"
    )
    if fayl_nomi:
        self.statusBar().showMessage(f"Saqlandi: {fayl_nomi}", 2000)

def oxirgi_ochish(self, n):
    self.statusBar().showMessage(f"document{n}.txt ochildi", 2000)

def toolbar_toggle(self):
    self.toolbar.setVisible(not self.toolbar.isVisible())

def statusbar_toggle(self):
    self.statusBar().setVisible(not self.statusBar().isVisible())

def tema_ozgartir(self, action):
    if action.text() == "Svetliy tema":
        self.setStyleSheet("background-color: #ffffff;")
    else:
        self.setStyleSheet("background-color: #2c3e50; color: white;")
    self.statusBar().showMessage(f"Tema: {action.text()}", 2000)

def haqida_korsot(self):
    QMessageBox.information(
        self, "Dastur haqida",
        "📄 NotePad v1.0\n\n"
        "PyQt5 bilan yozilgan sodda matn tahrirlash dasturi.\n"
        "Ctrl+N, Ctrl+O, Ctrl+S dan foydalaning!"
    )

if __name__ == "__main__":
    app = QApplication(sys.argv)
    oyna = NotePadApp()
    oyna.show()
    sys.exit(app.exec_())
```

10. Xulosa

QMenuBar → Menyu paneli (Fayl, Tahrirlash, Yordamasi...)
↓
QMenu → Ichki menyu (Yangi, Ochish, Saqlash...)
↓
QAction → Alohida harakat (Kesish, Nusxa, Yapistir...)
↓
triggered → Signal, slot ga bog'lanadi
↓
Funksiya chaqiriladi

Qo'shimchalar:

- ✓ setShortcut() → Ctrl+N, F5, Delete...
- ✓ setIcon() → Rasm yoki OS ikonasi
- ✓ addSeparator() → Chiziqli ajratgich
- ✓ Submenu → addMenu() bilan
- ✓ Checkbox → checkable=True
- ✓ RadioButton → QActionGroup bilan

💡 **Eslab qol:** QAction — bu universal! Menyu, toolbar, kontekst menyu da ishlashi mumkin.

● SAVOL 5:

PYQT5 DA MATN MUHARRIRI DASTURINI LOYIHALASH BOSQICHLARI: KERAKLI VIDJETLARNI TANLASH, LAYOUTLAR, FAYLLARNI OCHISH/SAQLASH FUNKSIYALARINI QO'SHISH VA DASTURNI TESTLASH.

1. Loyihalash Bosqichlari — Umumiy Ko'rinish

- 1-bosqich: Tahlil → Nima kerak? Qanday ko'rinadi?
 - 2-bosqich: Vidjetlar → Qaysi widgetlarni ishlatamiz?
 - 3-bosqich: Layout → Widgetlarni qanday joylashtiramiz?
 - 4-bosqich: Funksiyalar → Ochish, saqlash, tahrirlash
 - 5-bosqich: Testlash → Ishlayaptimi? Xatolar bormi?
-

2. Tahlil — Nima Kerak?

Matn muharriri dasturida quyidagilar bo'lishi kerak:

- ✓ Matn yozish maydoni
- ✓ Fayl ochish (Ctrl+O)
- ✓ Fayl saqlash (Ctrl+S)
- ✓ Yangi fayl (Ctrl+N)
- ✓ Nusxa/Kesish/Yapistirish
- ✓ Qaytarish/Qayta qilish (Undo/Redo)
- ✓ Qidiruv funksiyasi

- ✓ Hozirgi fayl nomi sarlavhada
- ✓ Status bar (qator/ustun raqami)

3. Vidjetlarni Tanlash

Matn muharriri uchun kerakli widgetlar:

QMainWindow	# Asosiy oyna (menu + toolbar + statusbar)
QPlainTextEdit	# Matn yozish maydoni (QTextEdit dan tezroq)
QMenuBar	# Menyu paneli (Fayl, Tahrirlash...)
QToolBar	# Asboblarni paneli (ikonkali tugmalar)
QStatusBar	# Pastki info paneli (qator raqami, fayl nomi)
QFileDialog	# Fayl ochish/saqlash dialogi
QMessageBox	# Xabar/ogohlantirish oynasi
QAction	# Menyu va toolbar harakatlari

Nima uchun QPlainTextEdit, QTextEdit emas?

QTextEdit	→ HTML, rasm, formatlash qo'llab-quvvatlaydi → Og'irroq, murakkab
QPlainTextEdit	→ Faqat oddiy matn → Tezroq, yengilroq ✓ → .txt fayllari uchun ideal

4. Layout Tuzilmasi

```
QMainWindow
├── QMenuBar          ← yuqorida (avtomatik)
│   ├── Fayl menyu
│   ├── Tahrirlash menyu
│   └── Yordamasi menyu
├── QToolBar          ← menubar ostida
│   ├── Yangi tugma
│   ├── Ochish tugma
│   └── Saqlash tugma
├── QPlainTextEdit    ← markaziy widget (eng katta)
└── QStatusBar        ← pastda (avtomatik)
    ├── Fayl nomi
    └── Qator : Ustun
```

Layout o'rnatish

```
class MatnMuharriri(QMainWindow):
    def __init__(self):
        super().__init__()

        # Markaziy widget – to'liq joy egallaydi
        self.text_area = QPlainTextEdit()
        self.text_area.setFont(QFont("Courier New", 12))
        self.setCentralWidget(self.text_area) # ← muhim!
```

```
# QMainWindow layout avtomatik:  
# MenuBar +ToolBar + CentralWidget + StatusBar
```

5. Menyu va Toolbar Yaratish

```
def menyu_yaratish(self):  
    menubar = self.menuBar()  
  
    # — FAYL MENYU —  
    fayl = menubar.addMenu("Fayl")  
  
    self.yangi_a = QAction("Yangi", self)  
    self.yangi_a.setShortcut("Ctrl+N")  
    self.yangi_a.triggered.connect(self.yangi_fayl)  
  
    self.ochish_a = QAction("Ochish", self)  
    self.ochish_a.setShortcut("Ctrl+O")  
    self.ochish_a.triggered.connect(self.fayl_ochish)  
  
    self.saqlash_a = QAction("Saqlash", self)  
    self.saqlash_a.setShortcut("Ctrl+S")  
    self.saqlash_a.triggered.connect(self.fayl_saqlash)  
  
    self.saqlash_as_a = QAction("Saqlash sifatida...", self)  
    self.saqlash_as_a.setShortcut("Ctrl+Shift+S")  
    self.saqlash_as_a.triggered.connect(self.fayl_saqlash_sifatida)  
  
    chiqish_a = QAction("Chiqish", self)  
    chiqish_a.setShortcut("Ctrl+Q")  
    chiqish_a.triggered.connect(self.chiqish)  
  
    fayl.addAction(self.yangi_a)  
    fayl.addAction(self.ochish_a)  
    fayl.addSeparator()  
    fayl.addAction(self.saqlash_a)  
    fayl.addAction(self.saqlash_as_a)  
    fayl.addSeparator()  
    fayl.addAction(chiqish_a)  
  
    # — TAHRIRLASH MENYU —  
    tahrirla = menubar.addMenu("Tahrirlash")  
  
    qaytarish_a = QAction("Qaytarish", self)  
    qaytarish_a.setShortcut("Ctrl+Z")  
    qaytarish_a.triggered.connect(self.text_area.undo)  
  
    qayta_a = QAction("Qayta qilish", self)  
    qayta_a.setShortcut("Ctrl+Y")  
    qayta_a.triggered.connect(self.text_area.redo)  
  
    kesish_a = QAction("Kesish", self)  
    kesish_a.setShortcut("Ctrl+X")  
    kesish_a.triggered.connect(self.text_area.cut)  
  
    nusxa_a = QAction("Nusxa", self)
```

```

nusxa_a.setShortcut("Ctrl+C")
nusxa_a.triggered.connect(self.text_area.copy)

yapistir_a = QAction("Yapistir", self)
yapistir_a.setShortcut("Ctrl+V")
yapistir_a.triggered.connect(self.text_area.paste)

tanlash_a = QAction("Hammasini tanlash", self)
tanlash_a.setShortcut("Ctrl+A")
tanlash_a.triggered.connect(self.text_area.selectAll)

tahrirla.addAction(qaytarish_a)
tahrirla.addAction(qayta_a)
tahrirla.addSeparator()
tahrirla.addAction(kesish_a)
tahrirla.addAction(nusxa_a)
tahrirla.addAction(yapistir_a)
tahrirla.addSeparator()
tahrirla.addAction(tanlash_a)

# — TOOLBAR —
toolbar = self.addToolBar("Asboblari")
toolbar.addAction(self.yangi_a)
toolbar.addAction(self.ochish_a)
toolbar.addAction(self.saqlash_a)

```

6. Fayl Ochish / Saqlash Funktsiyalari

```

def yangi_fayl(self):
    """Yangi fayl yaratish"""
    if self.text_area.document().isModified():
        javob = QMessageBox.question(
            self, "Saqlash",
            "O'zgarishlarni saqlaysizmi?",
            QMessageBox.Save | QMessageBox.Discard | QMessageBox.Cancel
        )
        if javob == QMessageBox.Save:
            self.fayl_saqlash()
        elif javob == QMessageBox.Cancel:
            return

    self.text_area.clear()
    self.joriy_fayl = None
    self.setWindowTitle("Yangi fayl — MatnMuharriri")
    self.statusBar().showMessage("Yangi fayl", 2000)

def fayl_ochish(self):
    """Faylni diskdan ochish"""
    fayl_nomi, _ = QFileDialog.getOpenFileName(
        self,
        "Faylni ochish",
        "",
        "Matn fayllar (*.txt);;Barcha fayllar (*)"
    )
    # boshlang'ich papka

```

```

if fayl_nomi:
    try:
        with open(fayl_nomi, 'r', encoding='utf-8') as f:
            self.text_area.setPlainText(f.read())
        self.joriy_fayl = fayl_nomi
        self.setWindowTitle(f"{fayl_nomi} – MatnMuharriri")
        self.statusBar().showMessage(f"Ochildi: {fayl_nomi}", 3000)
    except Exception as e:
        QMessageBox.critical(self, "Xato", f"Fayl ochilmadi:\n{e}")

def fayl_saqlash(self):
    """Joriy faylga saqlash"""
    if self.joriy_fayl:
        self._saqlash(self.joriy_fayl)
    else:
        self.fayl_saqlash_sifatida() # Yangi fayl → dialog ochiladi

def fayl_saqlash_sifatida(self):
    """Yangi nom bilan saqlash"""
    fayl_nomi, _ = QFileDialog.getSaveFileName(
        self,
        "Saqlash sifatida",
        "",
        "Matn fayllar (*.txt);;Barcha fayllar (*)"
    )
    if fayl_nomi:
        self._saqlash(fayl_nomi)

def _saqlash(self, fayl_nomi):
    """Ichki saqlash funksiyasi"""
    try:
        with open(fayl_nomi, 'w', encoding='utf-8') as f:
            f.write(self.text_area.toPlainText())
        self.joriy_fayl = fayl_nomi
        self.setWindowTitle(f"{fayl_nomi} – MatnMuharriri")
        self.text_area.document().setModified(False) # ← o'zgarish belgisi o'chadi
        self.statusBar().showMessage(f"Saqlandi: {fayl_nomi}", 3000)
    except Exception as e:
        QMessageBox.critical(self, "Xato", f"Saqlab bo'lmadi:\n{e}")

def chiqish(self):
    """Dasturdan chiqish"""
    if self.text_area.document().isModified():
        javob = QMessageBox.question(
            self, "Chiqish",
            "Saqlanmagan o'zgarishlar bor. Chiqasizmi?",
            QMessageBox.Save | QMessageBox.Discard | QMessageBox.Cancel
        )
        if javob == QMessageBox.Save:
            self.fayl_saqlash()
        elif javob == QMessageBox.Cancel:
            return
    QApplication.quit()

```

7. Status Bar — Qator/Ustun Raqami

```
def status_bar_sozlash(self):  
    """Kursorning qator va ustun raqamini ko'rsatish"""  
    self.text_area.cursorPositionChanged.connect(self.kursor_pozitsiyasi)  
  
def kursor_pozitsiyasi(self):  
    kursor = self.text_area.textCursor()  
    qator = kursor.blockNumber() + 1      # 0 dan boshlanadi → +1  
    ustun = kursor.columnNumber() + 1  
    self.statusBar().showMessage(  
        f"Qator: {qator} | Ustun: {ustun}"  
    )
```

8. Testlash Bosqichlari

- ✓ 1. Yangi fayl
 - Ctrl+N bosing
 - Eski matn o'chishi kerak
 - Sarlavha "Yangi fayl" bo'lishi kerak
 - ✓ 2. Fayl saqlash
 - Bir nima yozing
 - Ctrl+S bosing
 - .txt fayl yaratilishi kerak
 - Disk dan tekshiring
 - ✓ 3. Fayl ochish
 - Ctrl+O bosing
 - Saqlangan faylni tanlang
 - Matn to'g'ri chiqishi kerak
 - ✓ 4. Saqlanmagan o'zgarish
 - Bir nima yozing (saqlamang)
 - Ctrl+N yoki chiqish bosing
 - "Saqlaysizmi?" dialogi chiqishi kerak
 - ✓ 5. Undo/Redo
 - Matn yozing
 - Ctrl+Z → matn qaytishi kerak
 - Ctrl+Y → qayta kelishi kerak
 - ✓ 6. Katta fayl
 - 10,000+ qatorli fayl oching
 - Sekin bo'lmasligi kerak
 - ✓ 7. UTF-8
 - O'zbekcha/ruscha matn yozing
 - Saqlang va qayta oching
 - Belgilar to'g'ri chiqishi kerak
-

9. To'liq Tayyor Dastur

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QFont, QKeySequence
from PyQt5.QtCore import Qt

class MatnMuharriri(QMainWindow):
    def __init__(self):
        super().__init__()
        self.joriy_fayl = None
        self.setWindowTitle("Yangi fayl – MatnMuharriri")
        self.setGeometry(100, 100, 900, 600)

        # Markaziy widget
        self.text_area = QPlainTextEdit()
        self.text_area.setFont(QFont("Courier New", 12))
        self.setCentralWidget(self.text_area)

        self.menyu_yaratish()
        self.status_bar_sozlash()

    def menyu_yaratish(self):
        menubar = self.menuBar()

        # FAYL
        fayl = menubar.addMenu("Fayl")
        self.yangi_a = QAction("Yangi", self, shortcut="Ctrl+N",
        triggered=self.yangi_fayl)
        self.ochish_a = QAction("Ochish", self, shortcut="Ctrl+O",
        triggered=self.fayl_ochish)
        self.saqlash_a = QAction("Saqlash", self, shortcut="Ctrl+S",
        triggered=self.fayl_saqlash)
        self.saqlash_as_a = QAction("Saqlash sifatida", self,
        shortcut="Ctrl+Shift+S", triggered=self.fayl_saqlash_sifatida)
        self.chiqish_a = QAction("Chiqish", self, shortcut="Ctrl+Q",
        triggered=self.chiqish)

        fayl.addAction(self.yangi_a)
        fayl.addAction(self.ochish_a)
        fayl.addSeparator()
        fayl.addAction(self.saqlash_a)
        fayl.addAction(self.saqlash_as_a)
        fayl.addSeparator()
        fayl.addAction(self.chiqish_a)

        # TAHRIRLASH
        tahrirla = menubar.addMenu("Tahrirlash")
        tahrirla.addAction(QAction("Qaytarish", self, shortcut="Ctrl+Z",
        triggered=self.text_area.undo))
        tahrirla.addAction(QAction("Qayta qilish", self, shortcut="Ctrl+Y",
        triggered=self.text_area.redo))
        tahrirla.addSeparator()
        tahrirla.addAction(QAction("Kesish", self, shortcut="Ctrl+X",
        triggered=self.text_area.cut))
        tahrirla.addAction(QAction("Nusxa", self, shortcut="Ctrl+C",
        triggered=self.text_area.copy))
```

```

        tahrirla.addAction(QAction("Yapistir", self, shortcut="Ctrl+V",
triggered=self.text_area.paste))
        tahrirla.addSeparator()
        tahrirla.addAction(QAction("Hammasini tanlash",self, shortcut="Ctrl+A",
triggered=self.text_area.selectAll))

# TOOLBAR
toolbar = self.addToolBar("Asboblar")
toolbar.addAction([self.yangi_a, self.ochish_a, self.saqlash_a])

def status_bar_sozlash(self):
    self.text_area.cursorPositionChanged.connect(self.kursor_pozitsiyasi)
    self.statusBar().showMessage("Tayyor")

def kursor_pozitsiyasi(self):
    kursor = self.text_area.textCursor()
    qator = kursor.blockNumber() + 1
    ustun = kursor.columnNumber() + 1
    self.statusBar().showMessage(f"Qator: {qator} | Ustun: {ustun}")

def yangi_fayl(self):
    if self.text_area.document().isModified():
        j = QMessageBox.question(self, "Saqlash",
                                "O'zgarishlarni saqlaysizmi?",
                                QMessageBox.Save | QMessageBox.Discard | QMessageBox.Cancel)
        if j == QMessageBox.Save: self.fayl_saqlash()
        elif j == QMessageBox.Cancel: return
    self.text_area.clear()
    self.joriy_fayl = None
    self.setWindowTitle("Yangi fayl – MatnMuharriri")

def fayl_ochish(self):
    nom, _ = QFileDialog.getOpenFileName(
        self, "Faylni ochish", "",
        "Matn fayllar (*.txt);;Barcha fayllar (*)")
    if nom:
        try:
            with open(nom, 'r', encoding='utf-8') as f:
                self.text_area.setPlainText(f.read())
            self.joriy_fayl = nom
            self.setWindowTitle(f"{nom} – MatnMuharriri")
        except Exception as e:
            QMessageBox.critical(self, "Xato", f"Ochilmadi:\n{e}")

def fayl_saqlash(self):
    if self.joriy_fayl: self._saqlash(self.joriy_fayl)
    else: self.fayl_saqlash_sifatida()

def fayl_saqlash_sifatida(self):
    nom, _ = QFileDialog.getSaveFileName(
        self, "Saqlash sifatida", "",
        "Matn fayllar (*.txt);;Barcha fayllar (*)")
    if nom: self._saqlash(nom)

def _saqlash(self, nom):
    try:
        with open(nom, 'w', encoding='utf-8') as f:

```

```

        f.write(self.text_area.toPlainText())
        self.joriy_fayl = nom
        self.setWindowTitle(f"{nom} – MatnMuharriri")
        self.text_area.document().setModified(False)
        self.statusBar().showMessage(f"Saqlandi: {nom}", 3000)
    except Exception as e:
        QMessageBox.critical(self, "Xato", f"Saqlanmadi:\n{e}")

def chiqish(self):
    if self.text_area.document().isModified():
        j = QMessageBox.question(self, "Chiqish",
                                "Saqlanmagan o'zgarishlar bor. Chiqasizmi?",
                                QMessageBox.Save | QMessageBox.Discard | QMessageBox.Cancel)
        if j == QMessageBox.Save: self.fayl_saqlash()
        elif j == QMessageBox.Cancel: return
    QApplication.quit()

def closeEvent(self, event):
    """Oyna X tugmasi bosilganda"""
    if self.text_area.document().isModified():
        j = QMessageBox.question(self, "Chiqish",
                                "Saqlanmagan o'zgarishlar bor. Chiqasizmi?",
                                QMessageBox.Save | QMessageBox.Discard | QMessageBox.Cancel)
        if j == QMessageBox.Save:
            self.fayl_saqlash()
            event.accept()
        elif j == QMessageBox.Discard:
            event.accept()
        else:
            event.ignore() # ← oyna yopilmaydi!
    else:
        event.accept()

if __name__ == "__main__":
    app = QApplication(sys.argv)
    oyna = MatnMuharriri()
    oyna.show()
    sys.exit(app.exec_())

```

10. Xulosa

Bosqich 1 → Tahlil	: Nima kerak ekanini aniqlash
Bosqich 2 → Vidjetlar	: QTextEdit, QMainWindow, QAction
Bosqich 3 → Layout	: setCentralWidget, MenuBar,ToolBar
Bosqich 4 → Funksiyalar	: ochish, saqlash, yangi fayl
Bosqich 5 → Testlash	: Barcha holatlarni tekshirish

Muhim nuanslar:

- ✓ isModified() → saqlanmagan o'zgarish bormi?
- ✓ closeEvent() → X tugmasi uchun ham tekshirish
- ✓ try/except → fayl xatoliklarini ushlab
- ✓ encoding=utf-8 → o'zbek/rus belgilari uchun

💡 **Eslab qol:** `closeEvent()` ni ham yozmasang, foydalanuvchi X tugmasi bilan chiqib ketib, saqlanmagan matnini yo'qotishi mumkin!